

计算机系统设计PA5实验报告

苏耀磊 2311727

一、实验目的和环境概述

- 掌握 PAL 中基于 binary scaling 的定点数模拟方法，理解 FLOAT 类型在整数加减、乘除及类型转换中的实现思路。
- 通过实现 F2int、int2F、F_mul_int、F_div_int、F_mul_F、F_div_F、f2F、Fabs 等函数，加深对定点数运算与浮点数近似表达的理解。
- 最终能够成功运行仙剑奇侠传，并完成战斗等依赖定点数运算的关键功能。

本次实验环境使用 vscode 连接 docker 容器，在 debian 中进行实验，实验指导书参考 ics2017。

同时因为是在容器里进行，不支持 perf 这样的性能分析工具，因此并未性能测试，但是可以成功运行仙剑奇侠传。

二、实验内容

- [计算机系统设计PA5实验报告](#)
 - [一、实验目的和环境概述](#)
 - [二、实验内容](#)
 - [三、实验过程](#)
 - [1.实现binary scaling](#)
 - [2.实现 shrd、shld](#)
 - [四、实验过程中的问题与解决方法](#)
-

三、实验过程

1.实现binary scaling

我们已经在 PA3 中把仙剑奇侠传运行起来了，但却不能战斗，这是因为还有一些浮点数相关的工作需要处理，根据指导书，我们需要完成 navy-apps/apps/pal/include/FLOAT.h 和 navy-apps/apps/pal/src/FLOAT/FLOAT.c 里的一些函数，我们补充一下。

首先是 FLOAT.h：

```

8  static inline int F2int(FLOAT a) {
9  |   return a / (1 << 16);
10 }
11
12 static inline FLOAT int2F(int a) {
13 |   return a * (1 << 16);
14 }
15
16 static inline FLOAT F_mul_int(FLOAT a, int b) {
17 |   return a * b;
18 }
19
20 static inline FLOAT F_div_int(FLOAT a, int b) {
21 |   assert(b != 0);
22 |   return a / b;
23 }

```

这几个函数实现的是整数与定点数之间的转换和运算，在这里，FLOAT 并不是真正的浮点数类型，而是使用整数来模拟浮点数的小数表示。代码采用高 16 位表示整数部分，低 16 位表示小数部分。

F2int 的作用是将定点数转换为普通整数。由于定点数的小数部分占用了低 16 位，因此只需要除以 $1 \ll 16$ ，就能够去掉小数部分，得到整数结果，相当于把定点数的小数部分舍弃掉。

int2F 的作用则正好相反，它用于把普通整数转换成定点数。由于定点数的小数部分需要占据低 16 位，因此整数在转换时需要左移 16 位，也就是乘以 $1 \ll 16$ 。

F_mul_int 用于实现定点数与整数相乘。因为整数本身没有小数位，所以这里不需要进行额外缩放，直接使用 $a * b$ 即可。

F_div_int 则用于实现定点数除以整数，首先确保除数不为 0，随后直接执行 a / b 。由于 a 本身已经是定点格式，因此直接除以普通整数后，结果仍然能够保持正确的定点数表示。

接下来是 FLOAT.c:

```

5  ▾ FLOAT F_mul_F(FLOAT a, FLOAT b) {
6  |   return (FLOAT)((int64_t)a * b) >> 16;
7  }

```

F_mul_F 实现的是两个定点数之间的乘法运算，由于两个 16.16 定点数相乘后，小数位会变成 32 位，因此结果会被额外放大 2 的 16 次方倍，因此函数最后需要右移 16 位，把多出来的小数缩放消掉，从而恢复正确的 16.16 格式。

```

9  ∨ FLOAT F_div_F(FLOAT a, FLOAT b) {
10     assert(b != 0);
11
12     uint64_t dividend = ((uint64_t)((int64_t)a < 0 ? -(int64_t)a : (int64_t)a) << 16;
13     uint64_t divisor = (uint64_t)((int64_t)b < 0 ? -(int64_t)b : (int64_t)b);
14     uint64_t quotient = 0;
15     uint64_t remainder = 0;
16
17     ∨ for (int i = 63; i >= 0; i--) {
18         remainder = (remainder << 1) | ((dividend >> i) & 1ULL);
19     ∨     if (remainder >= divisor) {
20         remainder -= divisor;
21         quotient |= 1ULL << i;
22     }
23     }
24
25     ∨ if ((a < 0) ^ (b < 0)) {
26         return (FLOAT)(-(int64_t)quotient);
27     }
28
29     return (FLOAT)quotient;
30 }

```

而 F_div_F 实现的是两个定点数之间的除法。

首先检查除数为 0 的情况，随后程序分别取出 a 和 b 的绝对值，并转换成无符号 64 位整数。其中与乘法不同的是，除法中如果直接执行 a / b ，结果会丢失小数精度，因此需要把被除数左移 16 位。

之后从高位到低位循环，每次先把 remainder 左移一位，并从 dividend 中取出当前位加入余数。然后判断余数是否大于等于除数：如果余数足够大，就减去除数；同时在 quotient 对应位置上设置 1。

最终循环结束后，quotient 中保存的就是定点数除法结果，由于前面为了方便运算使用了绝对值，因此最后还需要恢复结果符号。

接下来是 f2F(float a)：

```

32  FLOAT f2F(float a) {
33      /* You should figure out how to convert `a' into FLOAT without
34       * introducing x87 floating point instructions. Else you can
35       * not run this code in NEMU before implementing x87 floating
36       * point instructions, which is contrary to our expectation.
37       *
38       * Hint: The bit representation of `a' is already on the
39       * stack. How do you retrieve it to another variable without
40       * performing arithmetic operations on it directly?
41       */
42
43      union {
44          float f;
45          uint32_t u;
46      } v = { .f = a };
47
48      uint32_t sign = v.u >> 31;
49      uint32_t exp = (v.u >> 23) & 0xff;
50      uint32_t frac = v.u & 0x7fffff;
51
52      if (exp == 0xff) {
53          assert(0);
54          return 0;
55      }

```

```

56
57  if (exp == 0) {
58      return 0;
59  }
60
61  int64_t mant = (1LL << 23) | frac;
62  int shift = (int)exp - 134;
63  int64_t abs_value;
64
65  if (shift >= 0) {
66      abs_value = mant << shift;
67  }
68  else {
69      abs_value = mant >> (-shift);
70  }
71
72  if (sign) {
73      return (FLOAT)(-(int64_t)abs_value);
74  }
75
76  return (FLOAT)abs_value;
77  }

```

f2F 函数的作用是将 C 语言中的 float 浮点数转换为定点数 FLOAT。首先定义一个联合体 union，其中同时包含 float f 和 uint32_t u。由于联合体中的成员共享同一块内存，因此当 v.f = a 后，可以直接通过 v.u 读取浮点数在内存中的二进制表示。

随后程序开始解析浮点数的符号位、指数、尾数部分，并处理特殊情况。如果 exp == 0xff，说明当前浮点数是无穷大或者 NaN，直接触发断言失败。如果 exp == 0，说明当前数是非规格化数或者非常接近 0 的数，直接返回 0。

之后程序开始恢复浮点数真正有效数字。在 IEEE754 中，规格化浮点数的尾数实际上默认存在一个隐藏的最低位 1，因此代码通过 (1LL << 23) | frac 将隐藏位补回来，得到完整的 24 位有效数字，并保存到 mant 中。

随后程序计算偏移量并根据 shift 的正负决定进行左移还是右移，如果 shift >= 0，说明结果需要扩大，因此进行左移；如果 shift < 0，说明结果需要缩小，因此进行右移，经过位移之后，得到的 abs_value 就已经是对应

的定点数值。

最后，程序根据符号位决定结果是否需要取负。

接下来是 Fabs：

```
79  ∨ FLOAT Fabs(FLOAT a) {  
80  |     return a < 0 ? -a : a;  
81  | }
```

直接通过判断 $a < 0$ 来决定是否返回 $-a$ 。

2.实现 shrd、shld

在执行 make run 的时候，发现还有指令没实现，在 nemu/src/cpu/exec/logic.c 里，我们补全如下：

```
107  ∨ make_EHelper(shrd) {  
108  |     rtl_andi(&t0, &id_src->val, 0x1f);  
109  |     if (t0 == 0) {  
110  |         print_asm_template2(shrd);  
111  |         return;  
112  |     }  
113  |  
114  |     uint32_t bits = id_dest->width * 8;  
115  |     uint32_t mask = (bits == 32 ? 0xfffffffffu : ((1u << bits) - 1));  
116  |     uint32_t dest = id_dest->val & mask;  
117  |     uint32_t src = id_src2->val & mask;  
118  |     uint32_t n = t0;  
119  |     uint64_t merged = ((uint64_t)src << bits) | dest;  
120  |     uint32_t res = (uint32_t)((merged >> n) & mask);  
121  |  
122  |     rtl_li(&t2, res);  
123  |     operand_write(id_dest, &t2);  
124  |     rtl_update_ZFSF(&t2, id_dest->width);  
125  |  
126  |     print_asm_template2(shrd);  
127  | }
```

首先是 shrd 双精度右移，首先取出实际移位位数，如果移位数为 0，则不进行任何操作。随后程序根据操作数宽度计算有效位数和掩码，并分别取出目标操作数 dest 和源操作数 src。接着把 src 放在高位、dest 放在低位，拼接成一个更大的整体数据，然后整体右移，这样在 dest 右移时，空出来的高位就会自动由 src 中的低位补充，最后程序取出结果的低位部分写回目标操作数，并更新 ZF 和 SF 标志位。

```

129 make_EHelper(shld) {
130     rtl_andi(&t0, &id_src->val, 0x1f);
131     if (t0 == 0) {
132         print_asm_template2(shld);
133         return;
134     }
135
136     uint32_t bits = id_dest->width * 8;
137     uint32_t mask = (bits == 32 ? 0xffffffffu : ((1u << bits) - 1));
138     uint32_t dest = id_dest->val & mask;
139     uint32_t src = id_src2->val & mask;
140     uint32_t n = t0;
141     uint32_t res = (uint32_t)(((uint64_t)dest << n) | (src >> (bits - n))) & mask;
142
143     rtl_li(&t2, res);
144     operand_write(id_dest, &t2);
145     rtl_update_ZFSF(&t2, id_dest->width);
146
147     print_asm_template2(shld);
148 }

```

shld 整体流程与 shrd 类似，同样先处理移位位数和操作数宽度。不同的是，这里会对 dest 进行左移，而左移后空出来的低位则由 src 的高位补充，最终结果同样会写回目标操作数，并更新对应标志位。

然后在 nemu/src/cpu/exec/all-instr.h 里补充函数声明：

```

54 make_EHelper(shrd);
55 make_EHelper(shld);

```

并在 nemu/src/cpu/decode/decode.c 里补充一个译码函数：

```

261 make_DHelper(CL_G2E) {
262     decode_op_rm(eip, id_dest, true, id_src2, true);
263     id_src->type = OP_TYPE_REG;
264     id_src->reg = R_CL;
265     rtl_lr_b(&id_src->val, R_CL);
266 #ifdef DEBUG
267     sprintf(id_src->str, "%c1");
268 #endif
269 }

```

CL_G2E 为使用 CL 寄存器作为移位次数的指令完成译码，首先调用 decode_op_rm 解析普通操作数，其中 id_dest 是目标操作数，id_src2 是另一个源操作数，并手动构造 id_src，将其类型设置为寄存器类型，同时指定寄存器为 R_CL。

接着通过 rtl_lr_b 从 CL 寄存器中读取 8 位的值，并保存到 id_src->val 中，作为真正的移位次数。

然后在 nemu/include/cpu/decode.h 补充函数声明如下：

```

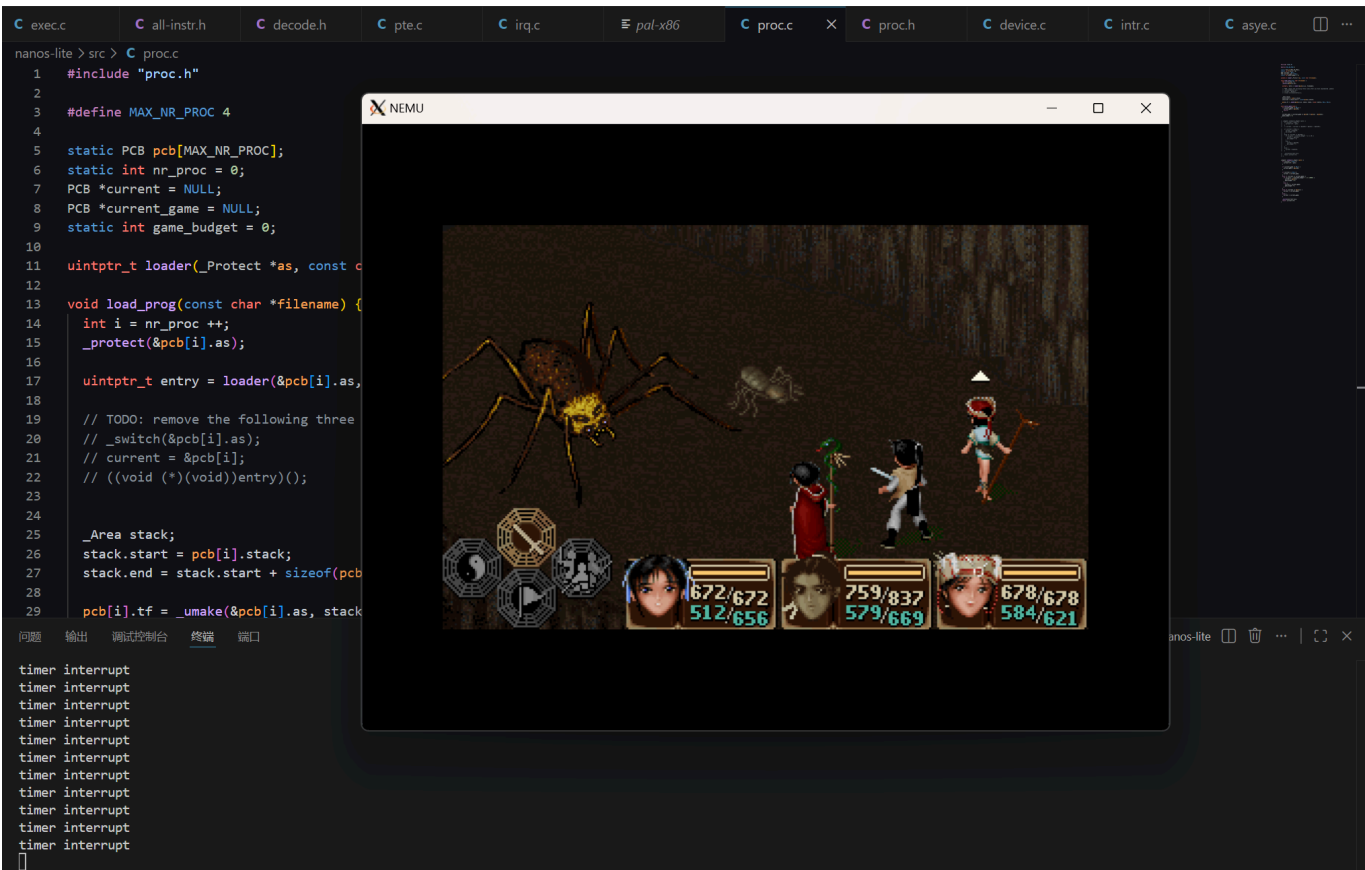
115 make_DHelper(mov_r2cr);
116 make_DHelper(mov_cr2r);

```

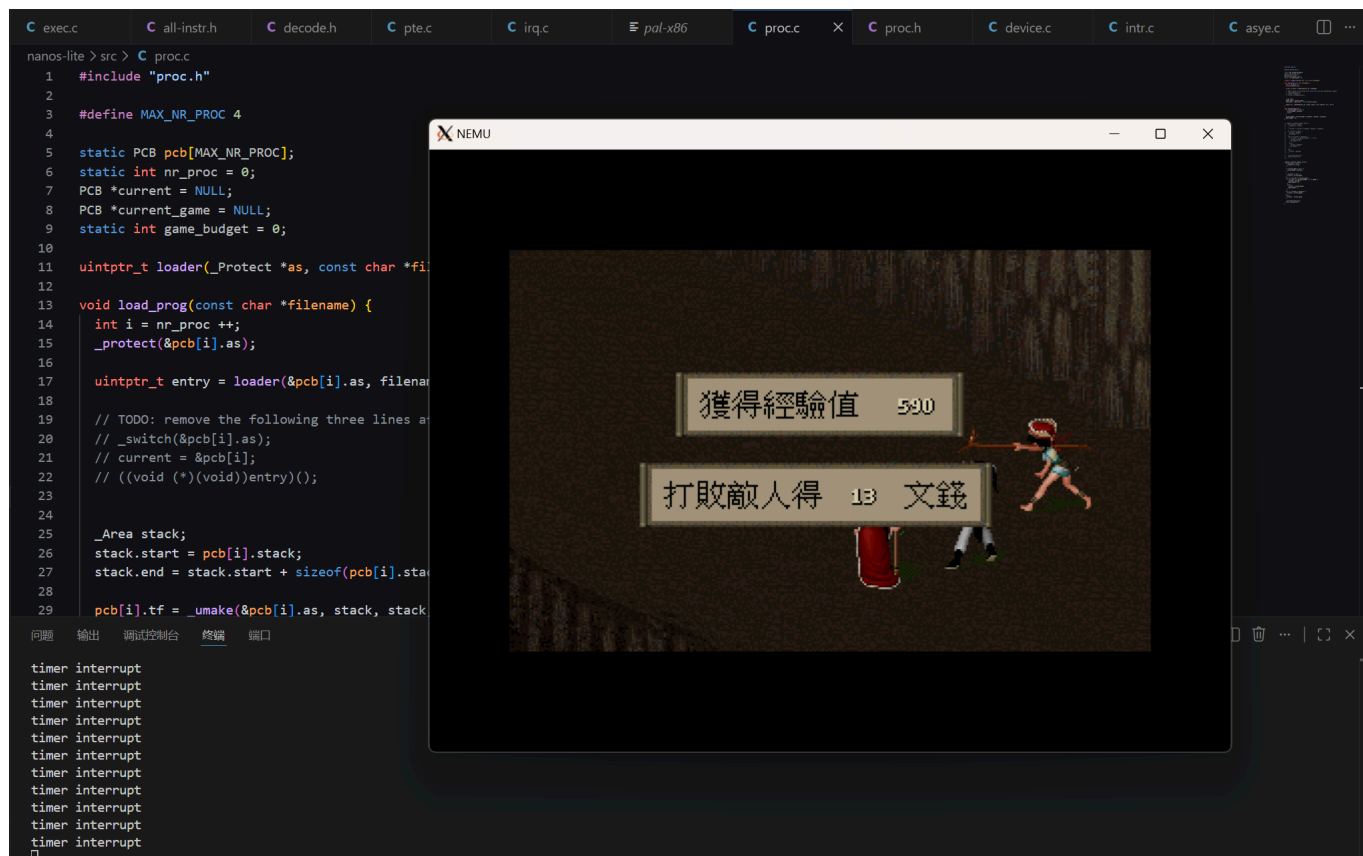
然后我们在 `nemu/src/cpu/exec/exec.c` 里实现对应内容：

```
185      /* 0xa4 */ IDEX(Ib_G2E, shld), IDEX(CL_G2E, shld), EMPTY, EMPTY,
186      /* 0xa8 */ EMPTY, EMPTY, EMPTY, EMPTY,
187      /* 0xac */ IDEX(Ib_G2E, shrd), IDEX(CL_G2E, shrd), EMPTY, IDEX(E2G, imul2),
```

这样我们就可以在仙剑奇侠传里战斗了，在 `/nanos-lite` 目录下执行 `make run`：



这蜘蛛居然打了两次才打过：



问题：比较FLOAT和float

FLOAT和float类型的数据都是32位, 它们都可以表示 2^{32} 个不同的数. 但由于表示方法不一样, FLOAT和float能表示的数集是不一样的. 思考一下, 我们用FLOAT来模拟表示float, 这其中隐含着哪些取舍?

float 使用的是 IEEE754 浮点数表示法, 而 FLOAT 使用的是定点数表示法, 浮点数由于具有指数部分, 因此小数点的位置可以动态变化, 这使得 float 能够同时表示非常大的数和非常小的数, 动态范围非常广。而定点数的小数点位置固定, 没有指数部分, 因此可表示的范围会受到很大限制。

浮点数的精度是不均匀分布的, 越接近 0 的区域精度越高, 而数值越大, 相邻两个可表示数之间的间隔也会变大, 而定点数由于小数位固定, 因此它在整个范围内的小数精度基本一致, 因此它在固定范围内精度稳定, 但无法像浮点数那样同时兼顾大范围与高精度。

此外, 定点数在运算过程中也更容易出现精度损失, 由于很多小数无法被固定的小数位精确表示, 因此在乘法和除法过程中经常需要截断或移位, 这会不断累积误差。而浮点数在很多情况下能够更自然地保持有效数字。

四、实验过程中的问题与解决方法

首先是在运行仙剑奇侠传时, 多次出现 invalid opcode 异常, 然后通过反汇编定位发现, PAL 中使用了 shld 和 shrd 等双字移位指令, 而 NEMU 原先并未完整支持这些指令的译码和执行流程, 只能手动在 NEMU 中补充对应的指令实现。

在实现定点数运算时, 最开始采用了较为复杂的 64 位中间运算写法, 但当前实验环境下编译器并不支持 `__int128`, 编译会产生错误, 最后改为使用 64 位整数和手写移位、除法逻辑实现 binary scaling 下的浮点模拟。

在实现 `F_mul_F`、`F_div_F`、`f2F` 等函数时，遇到了 64 位整除会被编译器自动转换为 `__divdi3` 的问题，最终导致链接失败，最后的解决方案是避免依赖运行库中的 64 位除法实现，改用手写的移位除法逻辑完成定点数除法，总体来说，最后一次 PA 还是让我收获颇丰。